Lab 2 — Variables & Reusable Modules

Terraform Course · Week 12 · Duration: 2 hours

Table of Contents

Learning Objectives 1
Before You Start 1
Part 1 — Input Variables (25 min) 2
Part 2 — Input Validation (15 min) 4
Part 3 — Locals (15 min) 4
Part 4 — Outputs (15 min) 5
Part 5 — Refactor Into a Module (35 min) 6
Part 6 — Use a Registry Module (20 min) 10
Part 7 — Clean Up (5 min) 11
Deliverables 11
Grading Rubric 11

Learning Objectives

By the end of this lab you will be able to:

1. Declare typed input variables with defaults, descriptions, and sensitive marking.
2. Use locals to remove repetition and improve readability.
3. Declare outputs, including sensitive outputs, to expose values from a configuration.
4. Use the full Terraform type system: string, number, bool, list, set, map, object.
5. Enforce rules on inputs with `validation` blocks.
6. Refactor a working configuration into a reusable local module and call it from a root module.
7. Consume a public registry module and pass its outputs into your own module.

Before You Start

Checklist

- ☐ Lab 1 completed and committed
- ☐ All Lab 1 AWS resources destroyed

- ☐ `terraform version` ≥ 1.5

Starting point

Copy your Lab 1 work as the starting point for Lab 2:

Variables (25 min)

Step 1.1 — Create variables.tf

Create a new file `variables.tf`:

```
variable "aws_region" {
  type        = string
  description = "AWS region to deploy into"
  default     = "us-east-1"
}

variable "project_name" {
  type        = string
  description = "Short name used as a prefix for all resource names"
  default     = "ipa-lab02"
}

variable "environment" {
  type        = string
  description = "Deployment environment"
  default     = "dev"
}

variable "instance_type" {
  type        = string
  description = "EC2 instance size"
  default     = "t3.micro"
}

variable "instance_count" {
  type        = number
  description = "How many web servers to create"
  default     = 1
}

variable "enable_monitoring" {
  type        = bool
  description = "Enable detailed CloudWatch monitoring"
  default     = false
}

variable "allowed_http_cidrs" {
  type        = list(string)
  description = "CIDR blocks permitted to reach port 80"
  default     = ["0.0.0.0/0"]
}
```

```
variable "extra_tags" {
  type        = map(string)
  description = "Additional tags applied to every resource"
  default     = {}
}
```

Step 1.2 — Add a complex (object) type

Terraform's structural types let you group related settings. Add:

```
variable "root_volume" {
  type = object({
    size_gb   = number
    type      = string
    encrypted = bool
  })
  description = "Root EBS volume configuration"
  default = {
    size_gb   = 8
    type      = "gp3"
    encrypted = true
  }
}
```

Step 1.3 — Add a sensitive variable

```
variable "admin_password" {
  type        = string
  description = "Demonstration of a sensitive input - do NOT use in production"
  sensitive   = true
  default     = "ChangeMe123!"
}
```

> **Question 1:** After you apply, what does Terraform print where a sensitive value would appear? Is the value still stored in the state file? (Answer from the book, Chapter 3.3.)

Step 1.4 — Use the variables

Update `provider.tf` to use `var.aws_region`, and update `main.tf` to use `var.instance_type` and `var.allowed_http_cidrs`:

```
provider "aws" {
  region = var.aws_region
  # ... default_tags unchanged ...
}
ingress {
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = var.allowed_http_cidrs
}
```

Run `terraform init && terraform validate`.

Part 2 — Input Validation (15 min)

Step 2.1 — Restrict the environment name

Add a `validation` block inside `variable "environment"`:

```
variable "environment" {
  type        = string
  description = "Deployment environment"
  default     = "dev"

  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "The environment must be one of: dev, staging, prod."
  }
}
```

Step 2.2 — Restrict the instance type

```
variable "instance_type" {
  type        = string
  description = "EC2 instance size"
  default     = "t3.micro"

  validation {
    condition     = can(regex("^t[23]\\.", var.instance_type))
    error_message = "Only t2 and t3 instance families are permitted in this
course."
  }
}
```

Step 2.3 — Validate a numeric range

```
validation {
  condition     = var.instance_count >= 1 && var.instance_count <= 3
  error_message = "instance_count must be between 1 and 3."
}
```

Step 2.4 — Prove the validation works

```
terraform plan -var="environment=production"
terraform plan -var="instance_type=m5.large"
terraform plan -var="instance_count=99"
```

> **Checkpoint 1:** Capture the three error messages. Note at what stage validation fails — does Terraform contact AWS first?

Part 3 — Locals (15 min)

Locals are computed inside the configuration. They are for removing repetition, not for accepting user input.

Step 3.1 — Create locals.tf

```
locals {
  # A single naming prefix used everywhere
  name_prefix = "${var.project_name}-${var.environment}"

  # Tags merged from a fixed set plus whatever the caller supplies
  common_tags = merge(
    {
      Project     = var.project_name
      Environment = var.environment
      ManagedBy   = "Terraform"
      Owner       = "IPA-Student"
    },
    var.extra_tags
  )

  # Derived boolean - production gets stricter settings
  is_production = var.environment == "prod"
}
```

Step 3.2 — Apply the locals everywhere

Replace every hard-coded name and tag block:

```
resource "aws_security_group" "web" {
  name   = "${local.name_prefix}-web-sg"
  vpc_id = data.aws_vpc.default.id
  # ...
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-web-sg" })
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
  monitoring    = var.enable_monitoring

  root_block_device {
    volume_size = var.root_volume.size_gb
    volume_type = var.root_volume.type
    encrypted   = var.root_volume.encrypted
  }

  tags = merge(local.common_tags, { Name = "${local.name_prefix}-web" })
}
```

> **Question 2:** What is the difference between a local and a variable with a default? Give one situation where only a local will work.

Part 4 — Outputs (15 min)

Step 4.1 — Create outputs.tf

```
output "instance_id" {
  value       = aws_instance.web.id
```

```
  description = "ID of the web server instance"
}

output "public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IPv4 address of the web server"
}

output "public_dns" {
  value       = aws_instance.web.public_dns
  description = "Public DNS hostname of the web server"
}

output "web_url" {
  value       = "http://${aws_instance.web.public_dns}"
  description = "Ready-to-click URL for the web server"
}

output "security_group_id" {
  value       = aws_security_group.web.id
  description = "ID of the web security group"
}

output "admin_password" {
  value       = var.admin_password
  description = "Demonstration of a sensitive output"
  sensitive   = true
}
```

Step 4.2 — Apply and read the outputs

```
terraform apply
terraform output
terraform output public_ip
terraform output -raw web_url
terraform output -json | jq
```

> **Checkpoint 2:** What is printed for `admin_password` by `terraform output`? Now run `terraform output -raw admin_password`. What happens, and what does that tell you about how `sensitive` protects you?

Part 5 — Refactor Into a Module (35 min)

This is the core exercise of the week.

Step 5.1 — Create the module directory

```
mkdir -p modules/web-server
cd modules/web-server
touch main.tf variables.tf outputs.tf
cd ../..
```

A module is simply a directory of `.tf` files. The directory you run `terraform apply` in is the **root module**.

Step 5.2 — Define the module's interface (modules/web-server/variables.tf)

Think carefully: what should the caller be allowed to control?

```
variable "name_prefix" {
  type        = string
  description = "Prefix applied to all resource names"
}

variable "vpc_id" {
  type        = string
  description = "VPC in which to create the security group"
}

variable "instance_type" {
  type        = string
  description = "EC2 instance size"
  default     = "t3.micro"
}

variable "allowed_http_cidrs" {
  type        = list(string)
  description = "CIDR blocks permitted to reach port 80"
  default     = ["0.0.0.0/0"]
}

variable "root_volume" {
  type = object({
    size_gb   = number
    type      = string
    encrypted = bool
  })
  default = {
    size_gb   = 8
    type      = "gp3"
    encrypted = true
  }
}

variable "tags" {
  type        = map(string)
  description = "Tags applied to all resources"
  default     = {}
}
```

Note there is **no provider block** inside the module. Modules inherit the provider from the root module.

Step 5.3 — Move the resources (modules/web-server/main.tf)

Move the `data "aws_ami"`, `aws_security_group`, and `aws_instance` blocks here, replacing every `var.project_name`/`local.*` reference with the module's own variables:

```
data "aws_ami" "amazon_linux" {
  most_recent = true
```

```
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }
}

resource "aws_security_group" "web" {
  name        = "${var.name_prefix}-web-sg"
  description = "Allow HTTP inbound"
  vpc_id      = var.vpc_id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = var.allowed_http_cidrs
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(var.tags, { Name = "${var.name_prefix}-web-sg" })
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.amazon_linux.id
  instance_type          = var.instance_type
  vpc_security_group_ids = [aws_security_group.web.id]

  root_block_device {
    volume_size = var.root_volume.size_gb
    volume_type = var.root_volume.type
    encrypted   = var.root_volume.encrypted
  }

  user_data = <<-EOF
    #!/bin/bash
    dnf install -y nginx
    echo "<h1>${var.name_prefix}</h1>" > /usr/share/nginx/html/index.html
    systemctl enable --now nginx
  EOF

  tags = merge(var.tags, { Name = "${var.name_prefix}-web" })
}
```

Step 5.4 — Expose outputs (modules/web-server/outputs.tf)

A module with no outputs is nearly useless — outputs are how the caller gets values back.

```
output "instance_id" {
  value       = aws_instance.web.id
  description = "ID of the created instance"
}

output "public_ip" {
  value       = aws_instance.web.public_ip
  description = "Public IPv4 address"
}

output "public_dns" {
  value       = aws_instance.web.public_dns
  description = "Public DNS hostname"
}

output "security_group_id" {
  value       = aws_security_group.web.id
  description = "ID of the created security group"
}
```

Step 5.5 — Call the module from the root

Delete the moved resources from the root `main.tf` and replace them with a module call:

```
module "web_server" {
  source = "./modules/web-server"

  name_prefix        = local.name_prefix
  vpc_id             = data.aws_vpc.default.id
  instance_type      = var.instance_type
  allowed_http_cidrs = var.allowed_http_cidrs
  root_volume        = var.root_volume
  tags               = local.common_tags
}
```

Update the root `outputs.tf` to read from the module:

```
output "public_ip" {
  value = module.web_server.public_ip
}

output "web_url" {
  value = "http://${module.web_server.public_dns}"
}
```

Step 5.6 — Re-initialize and apply

Adding a module requires `terraform init` again, because Terraform must install it.

```
terraform init
terraform validate
terraform plan
terraform apply
```

> **Checkpoint 3:** In the plan output, what is the full resource address of the instance now? (Hint: it starts with `module.`) Compare it to the address in Lab 1.

> **Question 3:** Terraform will likely plan to destroy and recreate your resources even though the configuration is functionally identical. Why?

Part 6 — Use a Registry Module (20 min)

Step 6.1 — Find the module

Open https://registry.terraform.io and search for `terraform-aws-modules/vpc/aws`. Read its inputs and outputs.

Step 6.2 — Call it

Add to the root `main.tf`:

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"

  name = "${local.name_prefix}-vpc"
  cidr = "10.0.0.0/16"

  azs             = ["${var.aws_region}a", "${var.aws_region}b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]

  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = local.common_tags
}
```

Step 6.3 — Wire the VPC into your own module

Change the module call to use the new VPC instead of the default one:

```
module "web_server" {
  source = "./modules/web-server"

  name_prefix = local.name_prefix
  vpc_id      = module.vpc.vpc_id
  # ...
}
```

You will also need to add a `subnet_id` variable to your module and pass `module.vpc.public_subnets[0]`, then use it on the `aws_instance`.

```
terraform init   # downloads the registry module
terraform apply
```

> **Question 4:** Why is `version = "~> 5.0"` required for a registry module but not for a local `./modules/...` module?
>
> **Checkpoint 4:** Run `terraform providers` and `terraform state list`. How many resources did the VPC module create on your behalf?

Part 7 — Clean Up (5 min)

```
terraform destroy
```

Verify with `terraform state list` — it should return nothing.

Deliverables

Submit to google form for week 12 submission

1. Root module: `terraform.tf`, `provider.tf`, `main.tf`, `variables.tf`, `locals.tf`, `outputs.tf`

   Local module: `modules/web-server/{main.tf,variables.tf,outputs.tf}`

   Zip into one file to submit

2. `lab02-report.md` containing:
   - Answers to Questions 1–4
   - A short paragraph: "What did moving code into a module make easier, and what did it make harder?"

Grading Rubric

Criterion	Marks
Typed variables incl. object and sensitive, correctly used	15
Validation blocks working, errors captured	15
Locals used to remove repetition (naming + tags)	10
Outputs incl. a sensitive output	10
Local module correctly structured with clean input/output interface	25
Registry VPC module consumed and wired into own module	15
Report answers and module README	10
Total	**100**